

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1110

COURSE OUTCOME COVERED

CO5: Analyze object-oriented software using quality assurance and usability testing
Techniques (BL4, BL5)
Assessment Tool Weightage: 25%

QUESTIONS: 05

Total Marks:25

Annexure A

Error Analysis Task

Code of Conduct:

*I TEJASWINI M (192411079) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:
TEJASWINI M

RUBRICS FOR CALCULATION:

Error Analysis Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q1. Scenario: Online Banking System

Modules:

User Login, Fund Transfer, Account Verification

Question:

Write pseudocode for a quality assurance testing process that validates secure user authentication and fund transfer operations. Include input validation, transaction verification, exception handling, and test result logging.

PSEUDOCODE – START

Initialize TestLog

// 1. User Authentication

INPUT username, password

IF username is empty OR password is empty THEN

 Log "Authentication Failed: Invalid Input"

ELSE

 TRY

 Authenticate(username, password)

 IF authentication is successful THEN

 Log "User Login: PASS"

 ELSE

 Log "User Login: FAIL"

 ENDIF

 CATCH AuthenticationException

 Log "Authentication Exception"

 ENDTRY

ENDIF

// 2. Account Verification

INPUT accountNumber

IF accountNumber is invalid THEN

 Log "Account Verification: FAIL"

ELSE

 TRY

 VerifyAccount(accountNumber)

 IF account is valid THEN

 Log "Account Verification: PASS"

 ELSE

 Log "Account Verification: FAIL"

```

    ENDIF

    CATCH AccountException
        Log "Account Verification Exception"
    ENDTRY
ENDIF

// 3. Fund Transfer
INPUT receiverAccount, amount

IF receiverAccount is empty OR amount <= 0 THEN
    Log "Fund Transfer: FAIL - Invalid Input"
ELSE
    TRY
        VerifyAccount(receiverAccount)
        CheckBalance(amount)

        IF balance is sufficient THEN
            ProcessTransfer(receiverAccount, amount)

            IF transaction is successful THEN
                VerifyTransaction()
                Log "Fund Transfer: PASS"
            ELSE
                Log "Fund Transfer: FAIL"
            ENDIF
        ELSE
            Log "Fund Transfer: FAIL - Insufficient Balance"
        ENDIF
    ENDIF

    CATCH TransactionException
        Log "Transaction Exception"
    ENDTRY
ENDIF

// 4. Final Test Report
GenerateTestReport(TestLog)

END

```

Q2. Scenario: E-Commerce Shopping Platform

Features:

Product Search, Shopping Cart, Payment Gateway

Question:

Design pseudocode for a usability testing evaluation that measures customer interaction efficiency, including product search time, checkout completion time, and navigation complexity. Explain how the algorithm collects and analyzes user behavior data.

PSEUDOCODE – START

Initialize UserBehaviorLog

// 1. Product Search

StartTimer()

INPUT searchKeyword

SearchProduct(searchKeyword)

StopTimer()

searchTime = TimerValue()

Record("Product Search Time", searchTime)

// 2. Navigation Analysis

navigationSteps = 0

WHILE user is browsing

 navigationSteps = navigationSteps + 1

 Record("Navigation Step", navigationSteps)

END WHILE

// 3. Shopping Cart

AddProductToCart()

Record("Cart Action", "Product Added")

// 4. Checkout

StartTimer()

ProceedToCheckout()

INPUT customerDetails

ConfirmOrder()

StopTimer()

checkoutTime = TimerValue()

```
Record("Checkout Completion Time", checkoutTime)
```

// 5. Payment

```
ProcessPayment()
```

```
IF payment successful THEN  
    Record("Payment", "PASS")  
ELSE  
    Record("Payment", "FAIL")  
ENDIF
```

// 6. Usability Analysis

```
IF searchTime <= ExpectedSearchTime THEN  
    SearchEfficiency = "GOOD"  
ELSE  
    SearchEfficiency = "POOR"  
ENDIF
```

```
IF checkoutTime <= ExpectedCheckoutTime THEN  
    CheckoutEfficiency = "GOOD"  
ELSE  
    CheckoutEfficiency = "POOR"  
ENDIF
```

```
IF navigationSteps <= MaximumSteps THEN  
    NavigationComplexity = "LOW"  
ELSE  
    NavigationComplexity = "HIGH"  
ENDIF
```

// 7. Generate Report

```
GenerateUsabilityReport(  
    searchTime,  
    checkoutTime,  
    navigationSteps,  
    SearchEfficiency,  
    CheckoutEfficiency,  
    NavigationComplexity  
)
```

```
END
```

HOW DATA IS COLLECTED AND ANALYZED

- Product Search Time: Timer records the time from entering a search until results are displayed.

- Checkout Completion Time: Measures the time taken from checkout initiation to order confirmation.
- Navigation Complexity: Counts clicks/pages required to complete the task.
- Behavior Logging: Stores user actions, timings, and payment results.
- Analysis: Compares measured values with expected limits and identifies areas where users experience difficulty.

Q3. Scenario: Hospital Management System

Question:

Write pseudocode to generate and execute test cases for appointment scheduling functionality. Include validation of doctor availability, appointment conflicts, patient information accuracy, and system responses to invalid requests.

PSEUDOCODE –
START

Initialize TestCaseLog

// Test Case 1: Patient Information Validation

INPUT patientID, patientName

IF patientID is empty OR patientName is empty THEN

 Log "Patient Information: FAIL - Invalid Data"

ELSE

 Log "Patient Information: PASS"

ENDIF

// Test Case 2: Doctor Availability

INPUT doctorID, appointmentDate, appointmentTime

CheckDoctorAvailability(doctorID, appointmentDate, appointmentTime)

IF doctor is available THEN

 Log "Doctor Availability: PASS"

ELSE

 Log "Doctor Availability: FAIL - Doctor Not Available"

ENDIF

// Test Case 3: Appointment Conflict

CheckExistingAppointment(doctorID, appointmentDate, appointmentTime)

IF appointment already exists THEN

 Log "Appointment Conflict: FAIL - Time Slot Already Booked"

ELSE

 Log "Appointment Conflict: PASS"

ENDIF

// Test Case 4: Schedule Appointment

IF patient information is valid

 AND doctor is available

 AND no appointment conflict THEN


```
ScheduleAppointment()
Log "Appointment Scheduling: PASS"

ELSE
    Log "Appointment Scheduling: FAIL"
ENDIF

// Test Case 5: Invalid Request
IF doctorID is invalid OR appointmentDate is invalid THEN
    Display "Invalid Appointment Request"
    Log "Invalid Request Handling: PASS"
ELSE
    Log "Invalid Request Handling: FAIL"
ENDIF

// Generate Test Report
GenerateTestReport(TestCaseLog)

END
```

WHAT IT TESTS

- Patient information accuracy → Checks valid patient details.
- Doctor availability → Ensures the doctor is free.
- Appointment conflict → Prevents double booking.
- Valid scheduling → Creates appointment only when all conditions are satisfied.
- Invalid requests → Displays an appropriate error response.
- Test logging → Records PASS/FAIL results for each test case.

Q4. Scenario: Airline Reservation System

Modules:

Passenger Registration, Flight Booking, Seat Allocation

Question:

Develop pseudocode for a constraint-based testing approach where booking requests must satisfy conditions such as valid passenger information, seat availability, and payment confirmation. Show how invalid booking scenarios are detected and handled.

PSEUDOCODE – START

Initialize TestCaseLog

INPUT passengerName, passengerID
INPUT flightID, seatNumber
INPUT paymentStatus

// 1. Validate Passenger Information

IF passengerName is empty OR passengerID is invalid THEN
 Display "Invalid Passenger Information"
 Log "Passenger Validation: FAIL"
ELSE
 Log "Passenger Validation: PASS"
ENDIF

// 2. Check Seat Availability

CheckSeatAvailability(flightID, seatNumber)

IF seat is not available THEN
 Display "Seat Not Available"
 Log "Seat Availability: FAIL"
ELSE
 Log "Seat Availability: PASS"
ENDIF

// 3. Verify Payment

IF paymentStatus = "Confirmed" THEN
 Log "Payment Confirmation: PASS"
ELSE
 Display "Payment Not Confirmed"
 Log "Payment Confirmation: FAIL"
ENDIF

// 4. Apply Booking Constraints

```
IF passenger information is valid  
  AND seat is available  
  AND payment is confirmed THEN
```

```
  ConfirmBooking()  
  AllocateSeat()  
  Display "Booking Successful"  
  Log "Booking: PASS"
```

```
ELSE  
  CancelBookingRequest()  
  Display "Booking Failed"  
  Log "Booking: FAIL"  
ENDIF
```

// 5. Generate Test Report

```
GenerateTestReport(TestCaseLog)
```

```
END
```

CONSTRAINTS TESTED

- Passenger information → Must be valid.
- Seat availability → Selected seat must be available.
- Payment confirmation → Payment must be successful.
- Booking → Confirmed only when all constraints are satisfied.
- Invalid scenarios → Booking is rejected and an appropriate error message is displayed.
- Test logging → PASS/FAIL results are recorded.

Q5. Scenario: Smart Library Management System

Features:

Book Search, Book Reservation, Borrow/Return Services

Question:

Write pseudocode for integrating quality assurance and usability testing where the system evaluates both functional correctness (reservation and borrowing operations) and user experience factors such as response time, ease of navigation, and accessibility.

PSEUDOCODE –

START

Initialize TestLog

// 1. Book Search – Usability Testing

StartTimer()

SearchBook(bookName)

StopTimer()

responseTime = TimerValue()

Log("Search Response Time", responseTime)

IF responseTime <= ExpectedTime THEN

 Log("Search Response: PASS")

ELSE

 Log("Search Response: FAIL")

ENDIF

// 2. Book Reservation – Functional Testing

IF book is available THEN

 ReserveBook(bookID)

 IF reservation is successful THEN

 Log("Book Reservation: PASS")

 ELSE

 Log("Book Reservation: FAIL")

 ENDIF

ELSE

 Display "Book Not Available"

 Log("Reservation Handling: PASS")

ENDIF

// 3. Borrow Book – Functional Testing

IF member is valid AND book is available THEN

 BorrowBook(bookID, memberID)

```
    Log("Borrow Operation: PASS")
ELSE
    Display "Borrowing Not Allowed"
    Log("Borrow Operation: FAIL")
ENDIF
```

```
// 4. Return Book – Functional Testing
ReturnBook(bookID)
```

```
IF return is recorded successfully THEN
    Log("Return Operation: PASS")
ELSE
    Log("Return Operation: FAIL")
ENDIF
```

```
// 5. Navigation Testing
navigationSteps = CountUserClicks()
```

```
IF navigationSteps <= MaximumSteps THEN
    Log("Navigation: EASY")
ELSE
    Log("Navigation: DIFFICULT")
ENDIF
```

```
// 6. Accessibility Testing
CheckFontSize()
CheckKeyboardNavigation()
CheckReadableLabels()
```

```
IF accessibility requirements are satisfied THEN
    Log("Accessibility: PASS")
ELSE
    Log("Accessibility: FAIL")
ENDIF
```

```
// 7. Generate QA & Usability Report
GenerateReport(
    TestLog,
    responseTime,
    navigationSteps
)

END
```

WHAT IS EVALUATED

- Functional correctness: Book reservation, borrowing, and return operations.
- Response time: Measures how quickly book search responds.
- Ease of navigation: Counts clicks/steps required to perform tasks.
- Accessibility: Checks readable text, labels, and keyboard navigation.
- QA logging: Records PASS/FAIL results and generates a final report.